pfi

Python for Fixed Income

COURSE 2 · BOND MATH

The Price-Yield Relationship

WEEK 1 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note: four prices, four places

The Desk Lead's Note

Notebook 02 priced the four: Pellwick above 100, Dunmarrow on it, Quorvane and Halvering below. Tell me why each sits where it does. Show me how each price moves as yields move, and which bond moves most. If yields never move, where does each price go? A dealer offers an annual-pay bond at 5.45 against Quorvane's 5.40: which yields more? Put the restatement in "bondmath", docstring first, prove it against Excel's EFFECT and NOMINAL, commit with a message that says why, and take it home.

The issuers are invented; every yield move is a what-if.

Four Bonds, Notebook 02's Prices

issuer	ticker	coupon_pct	years	yield_pct	price
Quorvane Energy	QVNE	5.250	5	5.400	99.3503
Dunmarrow Rail	DNMR	6.125	7	6.125	100.0000
Pellwick Paper	PLWK	4.750	3	4.600	100.4159
Halvering Foods	HLVF	7.500	10	7.700	98.6227

```
......   [100%]
6 passed in 0.50s
```

Four functions, six tests, four prices: the day starts where notebook 02 ended.

Four prices, four places against 100: today says why.

3.1 Par, premium, and discount

Course 1's Combining and Loading, Part 2 labelled a price premium, par, or discount by looking at the price. On a coupon date one comparison decides the label before any price is worked: the coupon against the yield.

Coupon Against Yield

```
bonds['coupon_minus_yield'] = bonds['coupon_pct'] - bonds['yield_pct']
rounded = bonds['price'].round(6)
bonds['label'] = np.where(rounded > 100, 'premium', np.where(rounded < 100, 'discount', 'par'))
table = bonds[['ticker', 'coupon_pct', 'yield_pct', 'coupon_minus_yield', 'price', 'label']]
formats = {'coupon_pct': '{:.3f}', 'yield_pct': '{:.3f}', 'coupon_minus_yield': '{:+.3f}',
           'price': '{:.4f}'}
table.style.format(formats).hide(axis='index')
```

ticker	coupon_pct	yield_pct	coupon_minus_yield	price	label
QVNE	5.250	5.400	-0.150	99.3503	discount
DNMR	6.125	6.125	+0.000	100.0000	par
PLWK	4.750	4.600	+0.150	100.4159	premium
HLVF	7.500	7.700	-0.200	98.6227	discount

In a format, "+" prints the sign of a positive number too: "{:+.3f}" shows "+0.150".

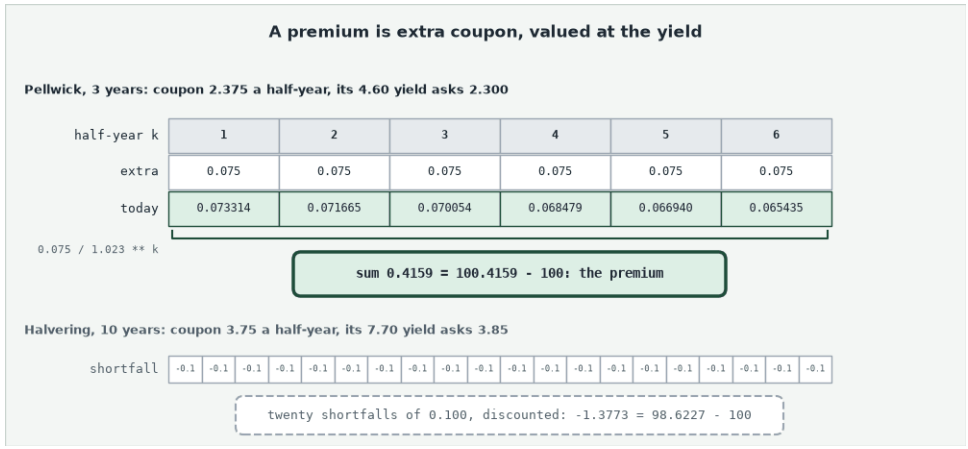
Coupon above the yield, premium; equal, par; below, discount.

Pellwick's coupon pays 4.75 / 2 = 2.375 a half-year, where its 4.60 yield asks only 2.300: an extra 0.075, six times. Notebook 02's "price_from_yield" values any coupon stream; give it the extra, coupon minus yield, as the coupon. Its price then holds the 100 as well, so take away the 100's share, notebook 01's "present_value".

```
bonds['extra_pv'] = [bondmath.price_from_yield(c - y, y, t) - bondmath.present_value(100, y, t)
                     for c, y, t in zip(bonds['coupon_pct'], bonds['yield_pct'], bonds['years'])]
assert (bonds['extra_pv'] - (bonds['price'] - 100)).abs().max() < 1e-9
bonds[['ticker', 'coupon_minus_yield', 'extra_pv']].round(6)
```

ticker	coupon_minus_yield	extra_pv
QVNE	-0.15	-0.649673
DNMR	0.00	0.000000
PLWK	0.15	0.415887
HLVF	-0.20	-1.377264

The extra coupon, valued at the yield



Pellwick's six extras are worth 0.415887 today: its premium, to the last digit. Halvering's twenty shortfalls cost 1.377264: its discount.

A coupon equal to the yield prices at 100 at every term, which is why a new bond's coupon is set near the yield: it comes to market near par.

With a last argument of 0, "-PV" values the payments alone, as notebook 02's coupons-alone row did.

Excel side by side

Job	Python	Excel	Result
The label, price in A2	`nested np.where`	`=IF(ROUND(A2,6)>100,"premium",IF(ROUND(A2,6)<100,"discount","par"))`	premium, par, discount for Pellwick, Dunmarrow, Quorvane
Pellwick's extra coupons	`extra_pv`	`=-PV(4.6/100/2,2*3,(4.75-4.6)/2,0)`	0.415887
Coupon at the yield, 30 years	`price_from_yield(6.125, 6.125, 30)`	`=-PV(6.125/100/2,2*30,6.125/2,100)`	100

A premium is extra coupon, valued at the yield; a discount is the shortfall. The coupon decides the side of 100, the term how far.

3.2 The price-yield curve: Excel's PRICE, row for row

Yields move, and prices follow. A desk counts the move in basis points: 100 bp is 1 point of yield.

Quorvane at Seven Yields

```
grid = pd.DataFrame({'yield_pct': [2.4, 3.4, 4.4, 5.4, 6.4, 7.4, 8.4]})
grid['price'] = [bondmath.price_from_yield(5.25, y, 5) for y in grid['yield_pct']]
grid['change'] = grid['price'] - bonds.loc[0, 'price']
formats = {'yield_pct': '{:.2f}', 'price': '{:.4f}', 'change': '{:+.4f}'}
grid.style.format(formats).hide(axis='index')
```

Excel's PRICE Column, Row for Row

```
tvm = pd.read_csv('bondmath_excel_reference_tvm.csv', index_col='case_id')
grid['excel'] = tvm.loc['tvm_price_01':'tvm_price_07', 'excel_value'].to_numpy()
gap = (grid['price'] - grid['excel']).abs().max()
assert gap < 1e-9
print(f'Seven PRICE rows, largest gap {gap:.1e} per 100 of face')
Seven PRICE rows, largest gap 5.7e-14 per 100 of face
```

Your module and Excel's "PRICE" give the same column: the largest gap is in the fourteenth decimal.

Excel side by side

Job	Excel
The column	yield_pct and price in A1:B1, 2.4 to 8.4 in A2:A8, =PRICE(DATE(2026,9,30),DATE(2031,9,30),5.25/100,A2/100,100,2,0) in B2, filled to B8
The chart	select A1:B8, then Insert, Scatter with Straight Lines

3.3 Coupon and maturity: which bond moves more

A steeper curve means a bigger move for the same 100 bp. Q. How far does each price move, in percent, when its yield falls or rises 100 bp?

```
terms = list(zip(bonds['coupon_pct'], bonds['yield_pct'], bonds['years']))
bonds['down'] = [bondmath.price_from_yield(c, y - 1, t) for c, y, t in terms]
bonds['up'] = [bondmath.price_from_yield(c, y + 1, t) for c, y, t in terms]
bonds['down_pct'] = (bonds['down'] - bonds['price']) / bonds['price'] * 100
bonds['up_pct'] = (bonds['up'] - bonds['price']) / bonds['price'] * 100
moves = bonds.sort_values('years')[['ticker', 'years', 'coupon_pct', 'down_pct', 'up_pct']]
formats = {'coupon_pct': '{:.3f}', 'down_pct': '{:+.2f}', 'up_pct': '{:+.2f}'}
moves.style.format(formats).hide(axis='index')
```

ticker	years	coupon_pct	up_pct
PLWK	3	4.750	-2.72
QVNE	5	5.250	-4.23
DNMR	7	6.125	-5.44
HLVF	10	7.500	-6.62

```
down_rows = ['tvm_price_03', 'tvm_price_08', 'tvm_price_10', 'tvm_price_12']
up_rows = ['tvm_price_05', 'tvm_price_09', 'tvm_price_11', 'tvm_price_13']
assert (bonds['down'] - tvm.loc[down_rows, 'excel_value'].to_numpy()).abs().max() < 1e-9
assert (bonds['up'] - tvm.loc[up_rows, 'excel_value'].to_numpy()).abs().max() < 1e-9
print('All eight moved prices match Excel')
All eight moved prices match Excel
```

"list()" keeps the pairs from "zip": a bare "zip" can be read only once, and two comprehensions read it.

Pellwick, 3 years, loses 2.72 percent for 100 bp up; Halvering, 10 years, 6.62. But the four differ in coupon and maturity at once, so move one lever at a time.

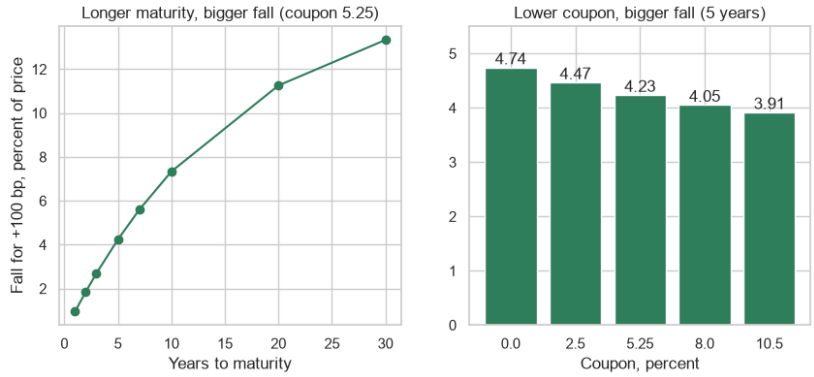
One lever at a time: maturity, then coupon

Quorvane's 5.25 Coupon and 5.40 Yield, Every Maturity

	0	1	2	3	4	5	6	7
years	1.00	2.00	3.00	5.00	7.00	10.00	20.00	30.00
price	99.86	99.72	99.59	99.35	99.14	98.85	98.18	97.78
up	98.90	97.87	96.91	95.14	93.59	91.60	87.13	84.75
up_pct	-0.95	-1.85	-2.70	-4.23	-5.59	-7.34	-11.26	-13.33

5 Years at 5.40, Every Coupon

	0	1	2	3	4
coupon_pct	0.0000	2.5000	5.2500	8.0000	10.5000
price	76.6118	87.4397	99.3503	111.2610	122.0889
up	72.9799	83.5346	95.1448	106.7550	117.3098
up_pct	-4.7407	-4.4660	-4.2330	-4.0499	-3.9144



Thirty years falls 13.33 percent, not thirty times the 1-year 0.95. The zero coupon falls most, 4.74 percent; the 10.50 coupon least, 3.91.

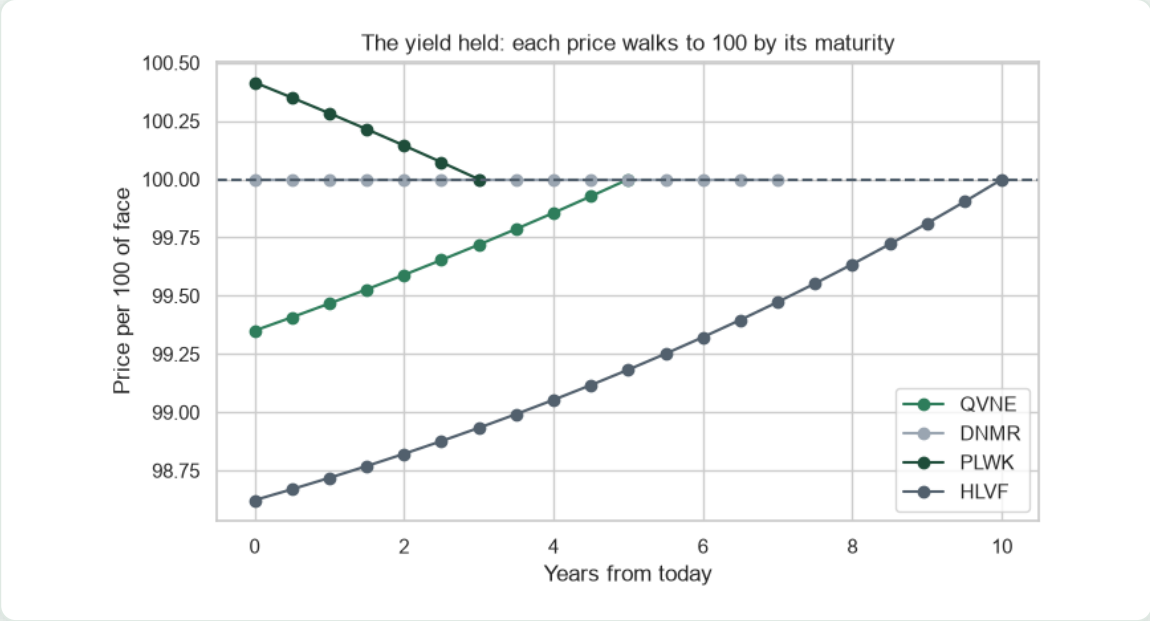
The zero coupon's 76.6118 is notebook 01's value of the 100 alone; 10.50's 122.0889 is notebook 02's Predict 3.

Job	Excel	Result
Halvering, 100 bp up	`=PRICE(DATE(2026,9,30),DATE(2036,9,30),7.5/100,8.7/100,100,2,0)`	92.092797
The change in percent	`today's PRICE (7.7/100) in B1, the 8.7/100 one in B2, =(B2-B1)/B1*100`	-6.62

A longer maturity and a lower coupon both mean a bigger move: both push the cash later. Notebook 09 puts one number on it.

3.4 Pull to par: the yield held, each price walks to 100

A bond repays 100 at maturity, whatever it costs today. Hold the yield, and the price must walk to 100: the pull to par.



Pellwick, the premium, walks down; Quorvane and Halvering walk up; Dunmarrow stays on the line. Every move on the chart is time passing: the yield never moved.

Halvering's Walk

```
for years_left in [10, 9.5, 5, 0.5]:
    walk_price = bondmath.price_from_yield(7.5, 7.7, years_left)
    print(f'{years_left:>4} years left: {walk_price:.4f}')
  10 years left: 98.6227
 9.5 years left: 98.6697
   5 years left: 99.1828
 0.5 years left: 99.9037
```

The first half-year adds 0.0470; the last, from 99.9037 to 100, adds 0.0963. The walk quickens: the nearer the price is to 100, the nearer each half-year's climb comes to the full shortfall of 0.100.

Excel side by side

Job	Excel
Halvering's walk	10 down to 0.5 in A1:A20,
	=-PV(7.7/100/2,2*A1,7.5/2,100) in B1, filled to B20
Pellwick's walk	3 down to 0.5 in A1:A6,
	=-PV(4.6/100/2,2*A1,4.75/2,100) in B1, filled to B6

3.5 One yield, three quotes

Quorvane's 5.40 compounds twice a year: 2.70 percent a half-year. The dealer's 5.45 compounds once a year.

Two yields compare only when they grow 100 to the same amount in a year.

```
growth = 100 * (1 + 5.4 / 100 / 2) ** 2
annual_pct = growth - 100
print(f'100 grows to {growth:.4f}: {annual_pct:.4f} percent compounded once a year')

quarterly_pct = ((growth / 100) ** (1 / 4) - 1) * 4 * 100
print(f'Quarterly: {quarterly_pct:.6f} percent')

dealer_semiannual = ((1 + 5.45 / 100) ** (1 / 2) - 1) * 2 * 100
print(f'Annual: Quorvane {annual_pct:.4f}, the dealer 5.4500')
print(f'Semiannual: Quorvane 5.4000, the dealer {dealer_semiannual:.4f}')
```

The second half-year earns interest on the first half-year's interest. Notebook 01's One Rate, Three Frequencies held the rate and let the growth change; here the growth holds and the rate changes.

Excel side by side. "EFFECT" restates a rate compounded "npery" times a year as an annual rate, and "NOMINAL" goes the other way; both take and return decimals.

Job	Excel
One year's growth	`=100*(1+5.4/100/2)^2`
The annual rate, written out	`=((1+5.4/100/2)^2-1)*100`
The annual rate, one function	`=EFFECT(5.4/100,2)`
Quarterly, chained	`=EFFECT(5.4/100,2) in A1, then =NOMINAL(A1,4)*100`
Back to semiannual	`=NOMINAL(A1,2)*100`

3.6 convert_yield, docstring first

```
%%writefile -a bondmath.py
def convert_yield(yield_pct: float, from_frequency: int, to_frequency: int) -> float:
    for frequency in (from_frequency, to_frequency):
        if frequency not in (1, 2, 4):
            raise ValueError(f"frequency must be 1, 2, or 4, not {frequency}")
    growth_one_year = (1 + yield_pct / 100 / from_frequency) ** from_frequency
    period_rate = growth_one_year ** (1 / to_frequency) - 1
    return period_rate * to_frequency * 100
```

The recipe never changes: one year's growth at the frequency you have, then the rate at the frequency you want. Docstring first, as in notebook 01: the units, percent in and percent out; the convention, the same growth in one year; and the Excel functions.

The loop checks both frequencies with the one test "cash_flows" uses.

Reload and Restate

ticker	annual_pct	yield_pct	quarterly_pct
DNMR	6.2188	6.125	6.0788
PLWK	4.6529	4.600	4.5738
HLVF	7.8482	7.700	7.6273

Halvering's annual rate sits 0.1482 above its quote and Pellwick's only 0.0529: the gap grows with the yield.

Two tests: Excel first, then the round trip

"EFFECT" rows go from "frequency" times a year to once: "convert_yield(rate, frequency, 1)". "NOMINAL" rows go the other way: "convert_yield(rate, 1, frequency)". Excel returns a decimal, so the expected value is Excel's times 100. The round trip proves the two directions undo each other, not that either is right, so the Excel test comes first.

```
%%writefile -a test_bondmath.py
def test_convert_yield_matches_excel_effect_and_nominal():
    for row in tvm_rows("EFFECT"):
        result = bondmath.convert_yield(float(row["rate_pct"]), int(row["frequency"]), 1)
        assert result == pytest.approx(float(row["excel_value"]) * 100, abs=1e-9), row["case_id"]
    for row in tvm_rows("NOMINAL"):
        result = bondmath.convert_yield(float(row["rate_pct"]), 1, int(row["frequency"]))
        assert result == pytest.approx(float(row["excel_value"]) * 100, abs=1e-9), row["case_id"]
def test_convert_yield_round_trip():
    quarterly = bondmath.convert_yield(6.0, 2, 4)
    assert bondmath.convert_yield(quarterly, 4, 2) == pytest.approx(6.0, abs=1e-12)
```

"8 passed": "convert_yield" agrees with Excel on all 22 rows.

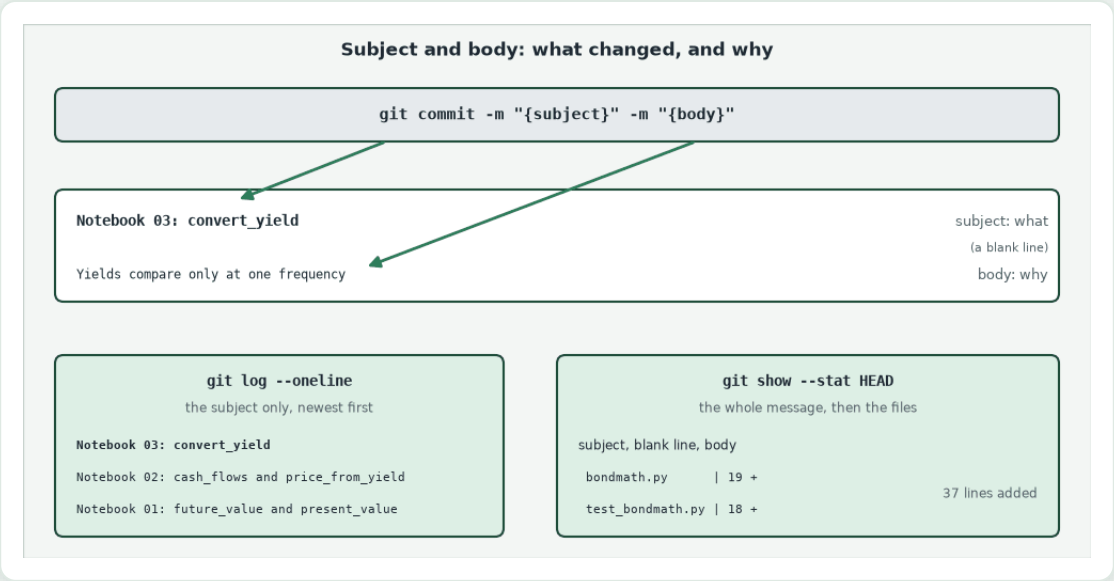
```
........   [100%]
8 passed in 0.50s
```

How Many Excel Numbers

```
16 EFFECT rows, 6 NOMINAL rows
Rates 0.5 to 10.25 percent; frequencies [1, 2, 4]
```

Excel side by side. Row "tvm_nominal_03" is "=NOMINAL(6.25/100,2)", 0.061553; the test compares "convert_yield(6.25, 1, 2)" with 6.1553.

3.7 A commit message that says why



In six months "git log" is how you will find a change. The first line, the subject, says what changed; the body says why. "git commit" takes one "-m" for each: the first is the subject, the second the body.

```
!git -C "{work}" diff --stat
 bondmath.py   | 19 ++++++++++++++++++
 test_bondmath.py | 18 ++++++++++++++++++
 2 files changed, 37 insertions(+)
```

```
subject = 'Notebook 03: convert_yield'
body = 'Yields compare only at one frequency'
!git -C "{work}" add bondmath.py test_bondmath.py
!git -C "{work}" commit -m "{subject}" -m "{body}"
!git -C "{work}" log --oneline
[main 4cdfc40] Notebook 03: convert_yield
 2 files changed, 37 insertions(+)
4cdfc40 Notebook 03: convert_yield
408a678 Notebook 02: cash_flows and price_from_yield
d029474 Notebook 01: future_value and present_value
```

"{subject}" and "{body}" put the two names' text into the command, as "{work}" does. "log --oneline" shows each subject alone, so the subject must say what on its own.

Three commits, each subject saying what changed; the body keeps the why for whoever reads it next.

Bring it home, and what you can now do

As in notebook 02, in a PowerShell terminal with the course's ".venv" active, with your own work folder. The copy replaces your two files: anything you added by hand shows in "git diff --stat" as "-" lines, to read before you commit.

```
$work = "$env:TEMP\pfi_course2_03_k4w7q2xr"
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\bondmath.py", "$work\test_bondmath.py" .
python -m pytest -q
git diff --stat
git add bondmath.py test_bondmath.py
git commit -m "Notebook 03: convert_yield" -m "Yields compare only at one frequency"
git log --oneline
git show --stat HEAD
```

Git then reports the commit, and "git log --oneline" lists three subjects, today's on top. "git show" opens in a pager when its output is long: press "q" to return to the prompt. In Colab, download the two ".py" files from the file browser on the left. The work folder can stay: a restart makes a new one.

"my_bondmath" holds five functions, eight tests, and three commits.

You Can Now

Say from the coupon and the yield whether a bond prices at a premium, at par, or at a discount, and value the premium as the coupon's excess.

Say which bond moves more for the same yield change: the longer one, and the one with the lower coupon.

Restate a yield at another frequency with "convert_yield", and in Excel with "EFFECT" and "NOMINAL".

Write a commit message with a subject that says what and a body that says why.

Predict first: a point of coupon, and 100 bp each way

Predict 1

Give Dunmarrow a coupon one point above its 6.125 yield: 7.125. Over 1 year that extra point is worth 0.9559. Keep the coupon and the yield; over 30 years, what is it worth? Write down a number.

```
for years in [1, 2, 3, 5, 7, 10, 20, 30]:
    premium = bondmath.price_from_yield(7.125, 6.125, years) - 100
    at_own_coupon = bondmath.price_from_yield(6.125, 6.125, years)
    print(f'{years:>2} years: premium {premium:.4f}; own coupon {at_own_coupon:.6f}')
```

Predict 2

From today, Quorvane's yield falls 100 bp; or, from today, it rises 100 bp. Compare the price's gain with its loss.

A) the gain and the loss are equal

B) the gain is larger

C) the loss is larger

```
grid = pd.DataFrame({'yield_pct': [2.4, 3.4, 4.4, 5.4, 6.4, 7.4, 8.4]})
grid['price'] = [bondmath.price_from_yield(5.25, y, 5) for y in grid['yield_pct']]
grid['change'] = grid['price'] - bonds.loc[0, 'price']
formats = {'yield_pct': '{:.2f}', 'price': '{:.4f}', 'change': '{:+.4f}'}
grid.style.format(formats).hide(axis='index')
```

Predict first: a half-year's return, and two quotes

Predict 3

Halvering's yield stays at 7.70. Over the next half-year it collects its 3.75 coupon and climbs 0.0470. What does it earn, as a percent of today's 98.6227? Write down a number.

```
bonds['pull'] = [bondmath.price_from_yield(c, y, t - 0.5) - p for c, y, t, p in
                 zip(bonds['coupon_pct'], bonds['yield_pct'], bonds['years'], bonds['price'])]
bonds['return_pct'] = (bonds['coupon_pct'] / 2 + bonds['pull']) / bonds['price'] * 100
assert (bonds['return_pct'] - bonds['yield_pct'] / 2).abs().max() < 1e-9
bonds[['ticker', 'coupon_pct', 'yield_pct', 'pull', 'return_pct']].round(6)
```

Predict 4

The dealer's annual-pay bond at 5.45, or Quorvane at 5.40 semiannual: which yields more?

A) the annual-pay bond: 5.45 is the bigger number

B) Quorvane

C) they yield the same

```
growth = 100 * (1 + 5.4 / 100 / 2) ** 2
annual_pct = growth - 100
print(f'100 grows to {growth:.4f}: {annual_pct:.4f} percent compounded once a year')
dealer_semiannual = ((1 + 5.45 / 100) ** (1 / 2) - 1) * 2 * 100
print(f'Annual: Quorvane {annual_pct:.4f}, the dealer 5.4500')
print(f'Semiannual: Quorvane 5.4000, the dealer {dealer_semiannual:.4f}')
```

The note, answered

The note asked	Answer
Why each sits there	coupon against yield; Pellwick's premium is its extra coupon, 0.4159
How prices move	along a curve: Quorvane +4.43 for 100 bp down, -4.21 for 100 bp up
Which moves most	Halvering, -6.62 percent for 100 bp up: the longest
If yields never move	each walks to 100 and earns its yield
5.45 annual or 5.40 semiannual	Quorvane: 5.4729 annual
In the module	convert_yield, 8 passed on 22 Excel rows
Saved and taken home	a commit that says why, in my_bondmath

Be proud: you read a quote the way a desk does, from the coupon, the yield, the term, and the frequency.

Next, Yield From Price: the arrow turns, from a quoted price back to its yield, where there is no formula to write.